- **流图表示法分析**
- **DAG 优化基本块(局部优化)**

今天主要围绕中间代码优化进行讲解，首先分析代码，然后进行优化。

- **流图表示法分析**

中间代码可使用流图来进行分析，以下是流图的定义。

基本块和流图

- 中间代码的**流图** (flow graph) 表示法
  - 中间代码划分成为**基本块** (basic block)
    - 控制流只能从基本块的第一条指令进入
    - 除基本块的最后一条指令外，控制流不会跳转/停机
  - 流图的**结点**是基本块，流图的**边**指明了哪些基本块可以跟在一个基本块之后运行

- 流图可以作为优化的基础
  - 它指出了基本块之间的控制流
  - 可以根据流图了解到一个值是否会被使用等信息

接着来看流图构造的算法。主要是确定 leaders：（1）第一个指令（2）跳转指令的目标指令（3）跳转指令的下一条指令（if else 一起看作是一个跳转指令）

划分基本块的算法

- 输入：三地址指令序列
- 输出：基本块的列表

- 方法
  - 确定**首指令**leader (基本块的第一个指令)
    - 第一个三地址指令
    - 任意一个 (条件或无条件) 转移指令的目标指令
    - 紧跟在一个 (条件或无条件) 转移指令之后的指令
  - 确定基本块
    - 每个首指令对应于一个基本块：从首指令开始到下一个首指令

然后以基本块为结点，可以构造边，例子为 ch p24。

流图的构造

- 流图的**结点**是基本块
  - 两个结点B和C之间有一条有向边 iff 基本块C的第一个指令可能在B的最后一个指令之后执行
- 存在边的原因
  - B的结尾指令是一条跳转到C的开头的条件/无条件语句
  - C紧跟在B之后，且B的结尾不是无条件跳转语句
  - 称B是C的**前驱** (predecessor)，C是B的**后继** (successor)
- **入口** (entry) /**出口** (exit) **结点**
  - 不和任何中间指令对应；入口到第一条指令有一条边；任何可能最后执行的基本块到出口有一条边

基于流图可以分析出各个变量是否活跃。注意：同一个变量在不同位置活跃状态不同，活跃状态取决于指定的活跃变量这一条件。

后续使用信息

- 变量值的**使用** (use)
  - 三地址语句i向变量x赋值，如果另一个语句j的运算分量为x，且从i开始有一条路径到达j，且路径上没有对x赋值，那么j就使用了i处计算得到的x的值
  - 我们说变量x在语句i后的程序点上**活跃** (live)
    - 程序执行完语句i时，x中存放的值将被后面的语句使用
    - **不活跃**是指变量的值不会被使用，而不是变量不会被使用
- 这些信息可以用于代码生成
  - 如果x在i处不活跃，且x占用了一个寄存器，我们可以把这个寄存器用于其它目的

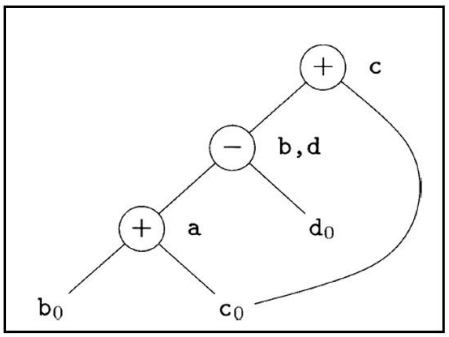
看个例子，ch8 p22。

- **DAG 优化基本块(局部优化)**

先通过一个例子来感受下，DAG 的构造。

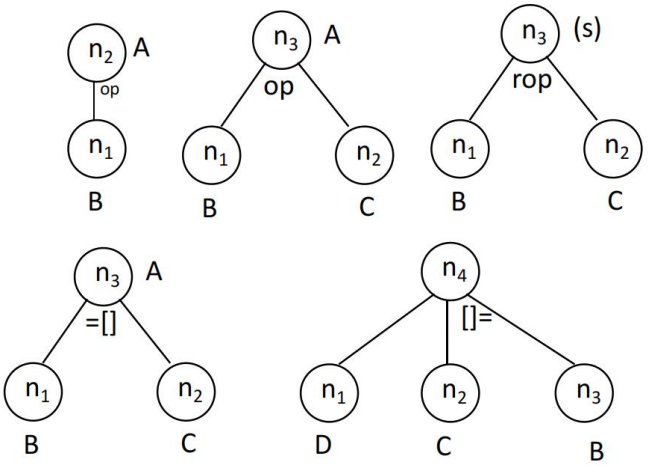
例子

- 根据DAG构造时结点产生的顺序
  - a = b + c
  - d = a – d
  - b = d
  - c = d + c



DAG 可以优化：（1）代数恒等式，如消除 x = x + 0, x = x * 1。（2）常量合并/折叠(constant folding) – 2 * 3.14 可以用 6.28 替换（3）消除死代码（4）消除局部公共子表达式（（5）强度消减(reduction in strength) – x2= x * x, 2 * x = x + x, x / 2 = x * 0.5）

基本组成部分有叶子（已知变量或常量）、中间节点（以运算符标识）、根，之间的连接方式有如下几种。若有多个变量，则孩子选择最近一次赋值的变量。



通过一个例子来看看 seuCP7 p30 p38